Practical -4

Source Code:

N-Queens Problem

```
def is_safe(board, row, col, n):
    # Check column and diagonals
    for i in range(row):
        if board[i] == col:
            return False

        if abs(board[i] - col) == abs(i - row):
            return False

    return True


def forward_check(board, row, col, n, domains):
    new_domains = [d.copy() for d in domains]

    for r in range(row + 1, n):
        # Same column
        if col in new_domains[r]:
            new_domains[r].remove(col)

        # Diagonal positions
        diag1 = col + (r - row)
        diag2 = col - (r - row)

        if diag1 in new_domains[r]:
            new_domains[r].remove(diag1)

        if diag2 in new_domains[r]:
            new_domains[r].remove(diag2)

    return new_domains


def solve_nqueens(board, row, n, domains):
    if row == n:
        return [board[:]]

    solutions = []
```

```
    for col in domains[row]:
        if is_safe(board, row, col, n):
            board[row] = col

            new_domains = forward_check(
                board, row, col, n, domains
            )

            if all(new_domains[r] for r in range(row + 1, n)):
                solutions += solve_nqueens(
                    board, row + 1, n, new_domains
                )

    return solutions


# Main
n = 8
domains = [list(range(n)) for _ in range(n)]

solutions = solve_nqueens(
    [-1] * n, 0, n, domains
)

print("Number of solutions:", len(solutions))
print("One solution:", solutions[0])
```

Output:

```
Number of solutions: 92
One solution: [0, 4, 7, 5, 2, 6, 1, 3]
```

Sudoku:

```
def find_unassigned(board):
    for i in range(9):
        for j in range(9):
            if board[i][j] == 0:
                return i, j

    return None, None
```

```
def is_valid(board, row, col, num):

    # Row and column check
    if num in board[row]:
        return False

    if num in [board[r][col] for r in range(9)]:
        return False

    # 3x3 subgrid check
    start_row = 3 * (row // 3)
    start_col = 3 * (col // 3)

    for r in range(start_row, start_row + 3):
        for c in range(start_col, start_col + 3):
            if board[r][c] == num:
                return False

    return True


def forward_checking(board, row, col):
    domain = {
        i: set(range(1, 10))
        for i in range(81)
    }

    for r in range(9):
        for c in range(9):
            if board[r][c] != 0:
                idx = r * 9 + c
                domain[idx] = {board[r][c]}

    return domain


def solve_sudoku(board):

    row, col = find_unassigned(board)

    if row is None:
        return True

    for num in range(1, 10):
```

```
        if is_valid(board, row, col, num):

            board[row][col] = num

            if solve_sudoku(board):
                return True

            board[row][col] = 0

    return False


# Example puzzle
sudoku_board = [
    [5, 3, 0, 0, 7, 0, 0, 0, 0],
    [6, 0, 0, 1, 9, 5, 0, 0, 0],
    [0, 9, 8, 0, 0, 0, 0, 6, 0],
    [8, 0, 0, 0, 6, 0, 0, 0, 3],
    [4, 0, 0, 8, 0, 3, 0, 0, 1],
    [7, 0, 0, 0, 2, 0, 0, 0, 6],
    [0, 6, 0, 0, 0, 0, 2, 8, 0],
    [0, 0, 0, 4, 1, 9, 0, 0, 5],
    [0, 0, 0, 0, 8, 0, 0, 7, 9]
]

solve_sudoku(sudoku_board)

for row in sudoku_board:
    print(row)
```

Output:

```
[5, 3, 4, 6, 7, 8, 9, 1, 2]
[6, 7, 2, 1, 9, 5, 3, 4, 8]
[1, 9, 8, 3, 4, 2, 5, 6, 7]
[8, 5, 9, 7, 6, 1, 4, 2, 3]
[4, 2, 6, 8, 5, 3, 7, 9, 1]
[7, 1, 3, 9, 2, 4, 8, 5, 6]
[9, 6, 1, 5, 3, 7, 2, 8, 4]
[2, 8, 7, 4, 1, 9, 6, 3, 5]
[3, 4, 5, 2, 8, 6, 1, 7, 9]
```

Map Colouring:

```
def is_safe(region, color, assignment, neighbors):
    for neighbor in neighbors[region]:
        if neighbor in assignment:
            if assignment[neighbor] == color:
                return False

    return True


def forward_check(assignment, domains, region, color, neighbors):
    new_domains = {
        r: set(domains[r])
        for r in domains
    }

    for neighbor in neighbors[region]:
        if color in new_domains[neighbor]:
            new_domains[neighbor].remove(color)

    return new_domains


def backtrack(assignment, domains, neighbors, colors):

    if len(assignment) == len(domains):
        return assignment

    region = next(
        r for r in domains
        if r not in assignment
    )

    for color in list(domains[region]):

        if is_safe(
            region,
            color,
            assignment,
            neighbors
        ):

            assignment[region] = color
```

```
            new_domains = forward_check(
                assignment,
                domains,
                region,
                color,
                neighbors
            )

            result = backtrack(
                assignment,
                new_domains,
                neighbors,
                colors
            )

            if result:
                return result

            del assignment[region]

    return None


# Example: Australia map
neighbors = {
    'WA': ['NT', 'SA'],
    'NT': ['WA', 'SA', 'Q'],
    'SA': ['WA', 'NT', 'Q', 'NSW', 'V'],
    'Q': ['NT', 'SA', 'NSW'],
    'NSW': ['Q', 'SA', 'V'],
    'V': ['SA', 'NSW'],
    'T': []
}

colors = ['Red', 'Green', 'Blue']

domains = {
    region: set(colors)
    for region in neighbors
}

solution = backtrack(
    {},
    domains,
```

```
    neighbors,
    colors
)

print("Map coloring solution:", solution)
```

Output:

```
Map coloring solution: {'WA': 'Green', 'NT': 'Blue', 'SA': 'Red', 'Q': 'Green', 'NSW':
'Blue', 'V': 'Green', 'T': 'Green'}
```